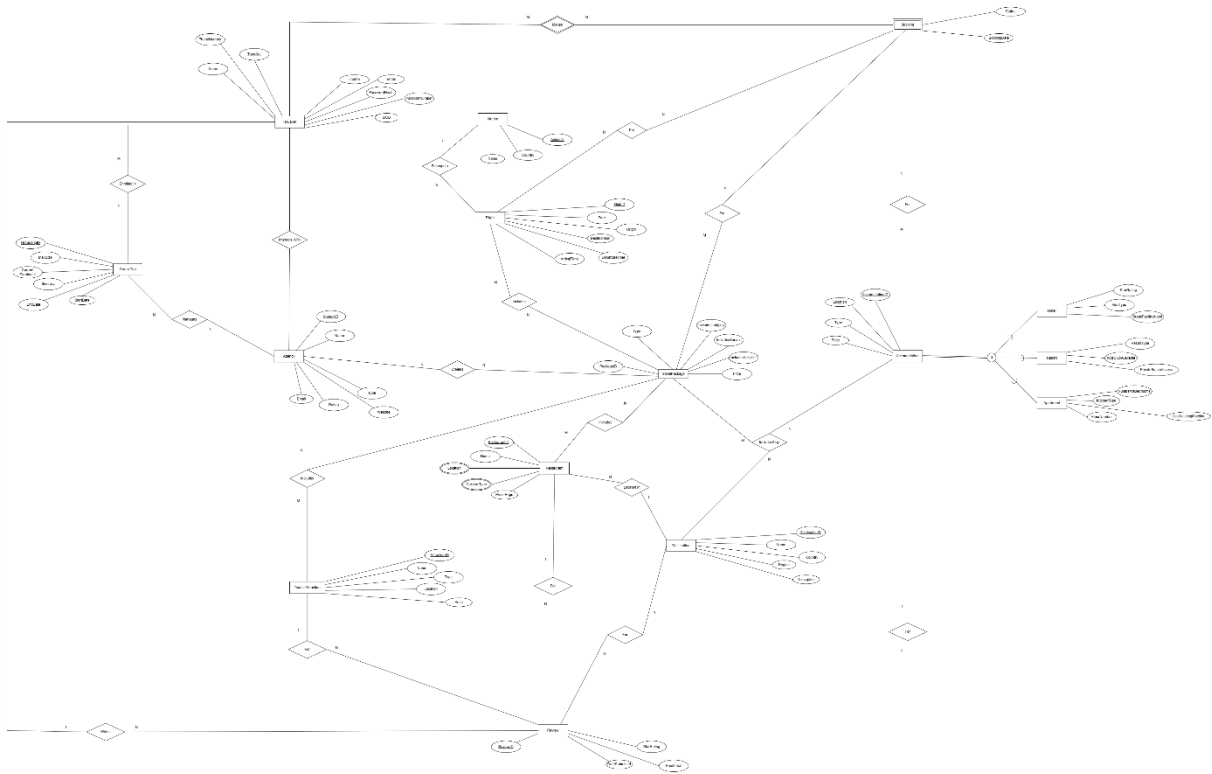


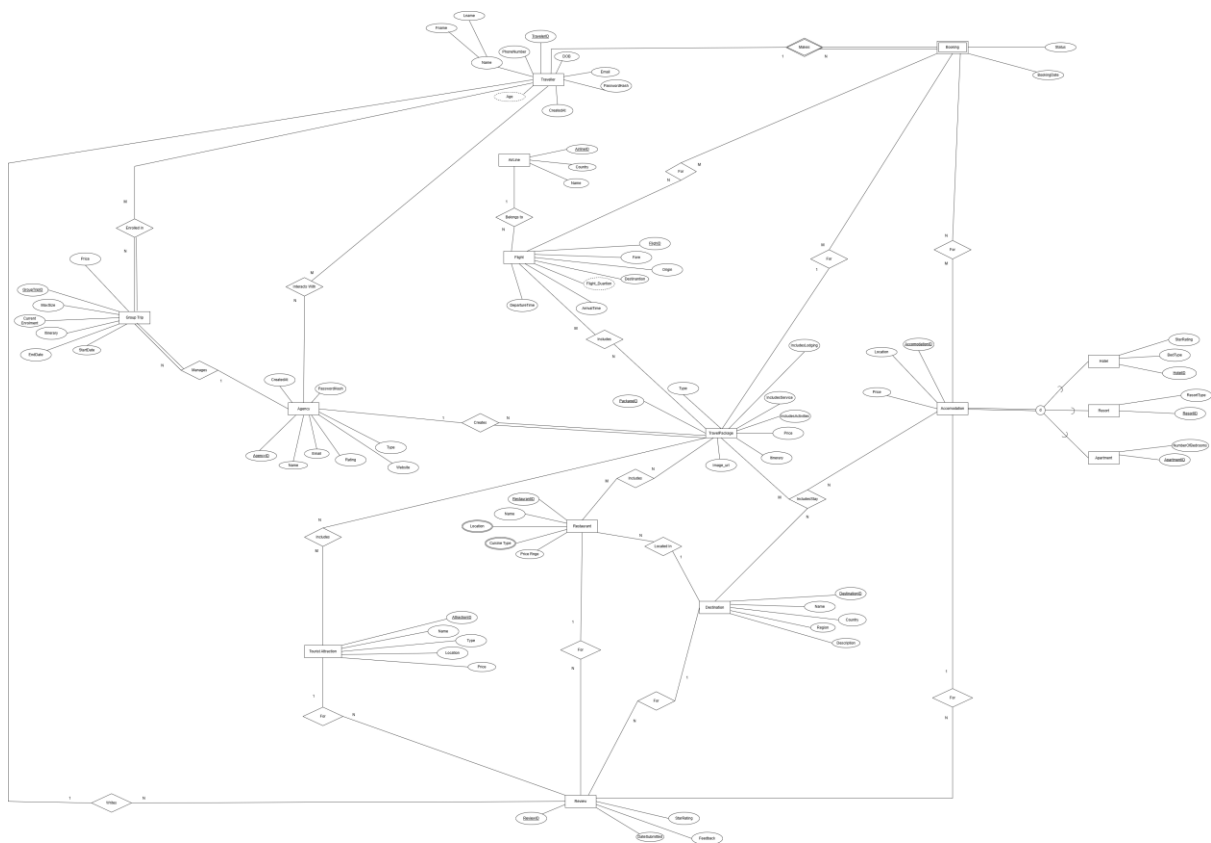
Task 2 : EER Diagram

The **first** iteration of EER Diagram.



The **final** iteration of the EER diagram.

The diagram was also not indicating the participation of a relationship and its entities. Added meaningful complex and derived attributes



Task 3: (E)ER Diagram to Relational Mapping

Step 1: Mapping Strong Entity Types

Each strong entity was converted into a relation with its own primary key.

Examples include:

- TRAVELLER
- AGENCY
- DESTINATION

- FLIGHT
- TRAVEL_PACKAGE
- BOOKING

Step 2: Mapping Weak Entity Types

The BOOKING entity was treated as a weak entity because it depends on a traveller and related travel services for its existence.

The relation was mapped as:

- BOOKING(BookingID, BookingDate, Status, TravellerID, PackageID, AccommodationID, FlightID)

TravellerID was included as a foreign key referencing TRAVELLER.

Assumption

A booking cannot exist without being associated with a traveller.

Step 3: Mapping Binary 1:1 Relationships

No mandatory 1:1 relationships were identified.

Step 4: Mapping Binary 1:N Relationships

For all 1:N relationships, the primary key from the “1” side was added as a foreign key to the relation on the “N” side.

Examples:

- AgencyID added to TRAVEL_PACKAGE
 - AirlineID added to FLIGHT
 - DestinationID added to ACCOMMODATION and RESTAURANT
 - TravellerID added to BOOKING
-

Step 5: Mapping Binary M:N Relationships

Many-to-many relationships were converted into junction tables with composite primary keys.

Examples:

- PACKAGE_INCLUDES_FLIGHT
- PACKAGE_INCLUDES_RESTAURANT
- ENROLLED_IN

Step 6: Mapping Multivalued Attributes

The multivalued restaurant location attribute was mapped into a separate relation:

- RESTAURANT_LOCATIONS(RestaurantID, Location)

Step 7: Mapping N-ary Relationships

The INCLUDES_STAY relationship between TRAVEL_PACKAGE, DESTINATION, and ACCOMMODATION was identified as an N-ary relationship.

A separate relation was created to represent this relationship:

- INCLUDES_STAY(PackageID, DestinationID, AccommodationID)

The foreign keys reference:

- TRAVEL_PACKAGE(PackageID)
- DESTINATION(DestinationID)
- ACCOMMODATION(AccommodationID)

The composite primary key consists of:

- PackageID
- DestinationID
- AccommodationID

Assumption

A travel package may include multiple accommodations across different destinations

Step 8: Mapping Specialisation and Generalisation

The ACCOMMODATION entity was specialized into:

- HOTEL
- RESORT
- APARTMENT

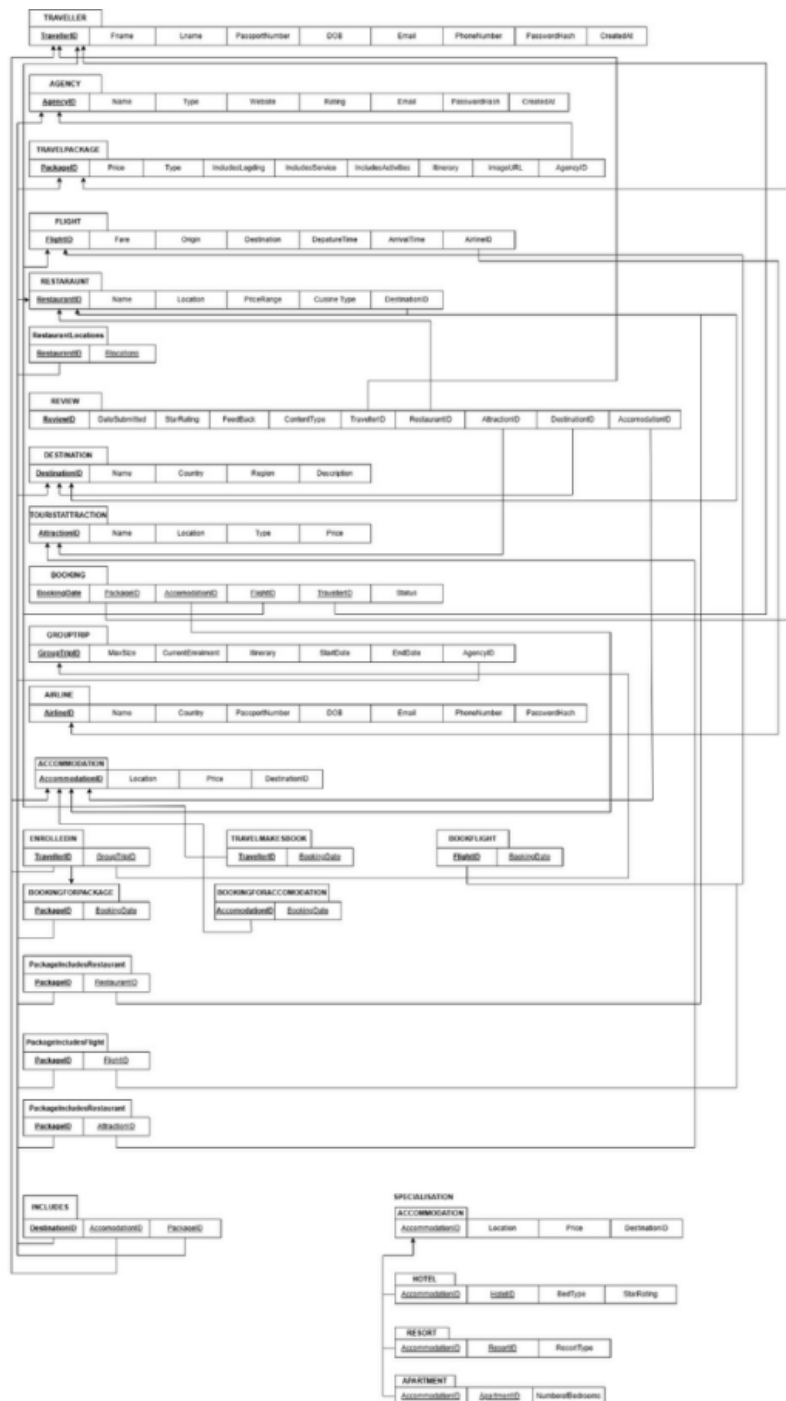
Option 8A (multiple relations: superclass and subclasses) was used because it avoids NULL values and keeps subtype-specific attributes separate.

Step 9: Mapping Unions

No unions/categories were identified in the EER diagram.

Assumptions and Design Choices

- Each travel package is created by one agency.
- Each flight belongs to one airline.
- One traveller can make multiple bookings.
- Junction tables were used for M:N relationships to maintain normalization and reduce redundancy.
- Separate subtype tables were used for accommodation specialization for better organization and scalability.



Task 4: Relational Schema

Database creation.

```
MariaDB [(none)]> create database tripistry;  
Query OK, 1 row affected (0.016 sec)
```

```
MariaDB [tripistry]> CREATE TABLE traveler (  
->     TravelerID INT AUTO_INCREMENT PRIMARY KEY,  
->     Name VARCHAR(100) NOT NULL,  
->     Email VARCHAR(150) NOT NULL UNIQUE,  
->     PhoneNumber VARCHAR(20) NOT NULL UNIQUE,  
->     PasswordHash VARCHAR(255) NOT NULL,  
->     DateOfBirth DATE,  
->     CreatedAt DATE NOT NULL DEFAULT (CURDATE())  
->     Age INT NOT NULL CHECK (Age > 0)  
-> );
```

```
MariaDB [tripistry]> CREATE TABLE agency (  
->     AgencyID INT AUTO_INCREMENT PRIMARY KEY,  
->     Name VARCHAR(100) NOT NULL UNIQUE,  
->     Website VARCHAR(200) NOT NULL,  
->     Type ENUM('Full Service', 'Boutique', 'OTA') NOT NULL,  
->     Rating DECIMAL(2,1) DEFAULT NULL,  
->     PasswordHash VARCHAR(255) NOT NULL,  
->     CreatedAt DATE NOT NULL DEFAULT (CURDATE())  
-> );
```

```
MariaDB [tripistry]> CREATE TABLE destination (  
->     DestinationID INT AUTO_INCREMENT PRIMARY KEY,  
->     Name VARCHAR(100) NOT NULL,  
->     Country VARCHAR(100) NOT NULL,  
->     Region VARCHAR(100) DEFAULT NULL,  
->     Description TEXT DEFAULT NULL  
-> );
```

```
MariaDB [tripistry]> CREATE TABLE airline (  
->     AirlineID INT AUTO_INCREMENT PRIMARY KEY,  
->     Name VARCHAR(100) NOT NULL,  
->     Country VARCHAR(100) DEFAULT NULL  
-> );
```

```
MariaDB [tripistry]> CREATE TABLE flight (  
-> FlightID INT AUTO_INCREMENT PRIMARY KEY,  
-> Origin VARCHAR(100) NOT NULL,  
-> Destination VARCHAR(100) NOT NULL,  
-> DepartureTime DATETIME NOT NULL,  
-> ArrivalTime DATETIME NOT NULL,  
-> Fare DECIMAL(10,2) NOT NULL,  
-> Flight_Duration TIME NOT NULL,  
-> AirlineID INT,  
-> FOREIGN KEY (AirlineID) REFERENCES airline(AirlineID) ON DELETE SET NULL  
-> );
```

```
MariaDB [tripistry]> CREATE TABLE accommodation (  
-> AccommodationID INT AUTO_INCREMENT PRIMARY KEY,  
-> Location VARCHAR(200) NOT NULL,  
-> Price DECIMAL(10,2) NOT NULL  
-> );
```

```
MariaDB [tripistry]> CREATE TABLE hotel (  
-> HotelID INT AUTO_INCREMENT PRIMARY KEY,  
-> AccommodationID INT NOT NULL UNIQUE,  
-> BedType VARCHAR(50) DEFAULT NULL,  
-> Stars INT DEFAULT NULL,  
-> FOREIGN KEY (AccommodationID) REFERENCES accommodation(AccommodationID) ON DELETE CASCADE  
-> );
```

```
MariaDB [tripistry]> CREATE TABLE resort (  
-> ResortID INT AUTO_INCREMENT PRIMARY KEY,  
-> AccommodationID INT NOT NULL UNIQUE,  
-> ResortType VARCHAR(100) DEFAULT NULL,  
-> FOREIGN KEY (AccommodationID) REFERENCES accommodation(AccommodationID) ON DELETE CASCADE  
-> );
```

```
MariaDB [tripistry]> CREATE TABLE apartment (  
-> ApartmentID INT AUTO_INCREMENT PRIMARY KEY,  
-> AccommodationID INT NOT NULL UNIQUE,  
-> Bedrooms INT DEFAULT NULL,  
-> FOREIGN KEY (AccommodationID) REFERENCES accommodation(AccommodationID) ON DELETE CASCADE  
-> );
```

```
MariaDB [tripistry]> CREATE TABLE touristattraction (  
-> AttractionID INT AUTO_INCREMENT PRIMARY KEY,  
-> Name VARCHAR(150) NOT NULL,  
-> Type VARCHAR(50) NOT NULL,  
-> Location VARCHAR(200) DEFAULT NULL,  
-> Price DECIMAL(10,2) DEFAULT 0,  
-> DestinationID INT,  
-> FOREIGN KEY (DestinationID) REFERENCES destination(DestinationID) ON DELETE CASCADE  
-> );
```

```
MariaDB [tripistry]> CREATE TABLE restaurant (  
-> RestaurantID INT AUTO_INCREMENT PRIMARY KEY,  
-> Name VARCHAR(150) NOT NULL,  
-> PriceRange VARCHAR(30) DEFAULT NULL,  
-> DestinationID INT,  
-> FOREIGN KEY (DestinationID) REFERENCES destination(DestinationID) ON DELETE CASCADE  
-> );
```

```
MariaDB [tripistry]> CREATE TABLE pack_cuisine (  
-> CuisineID INT AUTO_INCREMENT PRIMARY KEY,  
-> RestaurantID INT NOT NULL,  
-> Cuisine VARCHAR(100) NOT NULL,  
-> FOREIGN KEY (RestaurantID) REFERENCES restaurant(RestaurantID) ON DELETE CASCADE  
-> );
```

```
MariaDB [tripistry]> CREATE TABLE rest_location (  
-> LocationID INT AUTO_INCREMENT PRIMARY KEY,  
-> RestaurantID INT NOT NULL,  
-> Location VARCHAR(200) NOT NULL,  
-> FOREIGN KEY (RestaurantID) REFERENCES restaurant(RestaurantID) ON DELETE CASCADE  
-> );
```

```
MariaDB [tripistry]> CREATE TABLE travelpackage (  
-> PackageID INT AUTO_INCREMENT PRIMARY KEY,  
-> AgencyID INT NOT NULL,  
-> Name VARCHAR(200) NOT NULL,  
-> Type ENUM('Bundled', 'Individual') NOT NULL DEFAULT 'Bundled',  
-> Price DECIMAL(10,2) NOT NULL,  
-> IncludesLodging TINYINT(1) DEFAULT 0,  
-> IncludesService TINYINT(1) DEFAULT 0,  
-> IncludesActivities TINYINT(1) DEFAULT 0,  
-> DurationDays INT DEFAULT 0,  
-> ImageURL VARCHAR(500) DEFAULT NULL,  
-> FOREIGN KEY (AgencyID) REFERENCES agency(AgencyID) ON DELETE CASCADE  
-> );
```

```
MariaDB [tripistry]> CREATE TABLE includes (  
-> ID INT AUTO_INCREMENT PRIMARY KEY,  
-> PackageID INT NOT NULL,  
-> FlightID INT NOT NULL,  
-> FOREIGN KEY (PackageID) REFERENCES travelpackage(PackageID) ON DELETE CASCADE,  
-> FOREIGN KEY (FlightID) REFERENCES flight(FlightID) ON DELETE CASCADE  
-> );
```

```
MariaDB [tripistry]> CREATE TABLE package_attr (  
-> ID INT AUTO_INCREMENT PRIMARY KEY,  
-> PackageID INT NOT NULL,  
-> AttractionID INT NOT NULL,  
-> FOREIGN KEY (PackageID) REFERENCES travelpackage(PackageID) ON DELETE CASCADE,  
-> FOREIGN KEY (AttractionID) REFERENCES touristattraction(AttractionID) ON DELETE CASCADE  
-> );
```

```
MariaDB [tripistry]> CREATE TABLE package_rest (  
-> ID INT AUTO_INCREMENT PRIMARY KEY,  
-> PackageID INT NOT NULL,  
-> RestaurantID INT NOT NULL,  
-> FOREIGN KEY (PackageID) REFERENCES travelpackage(PackageID) ON DELETE CASCADE,  
-> FOREIGN KEY (RestaurantID) REFERENCES restaurant(RestaurantID) ON DELETE CASCADE  
-> );
```

```
MariaDB [tripistry]> CREATE TABLE packages (  
-> ID INT AUTO_INCREMENT PRIMARY KEY,  
-> TravelerID INT NOT NULL,  
-> PackageID INT NOT NULL,  
-> FOREIGN KEY (TravelerID) REFERENCES traveler(TravelerID) ON DELETE CASCADE,  
-> FOREIGN KEY (PackageID) REFERENCES travelpackage(PackageID) ON DELETE CASCADE  
-> );
```

```
MariaDB [tripistry]> CREATE TABLE booking (  
-> TravellerID INT NOT NULL,  
-> Status enum('Pending','Processing','Completed'),  
-> BookingDate DATE NOT NULL DEFAULT (CURDATE()),  
-> FOREIGN KEY (TravellerID) REFERENCES traveler(TravelerID) ON DELETE CASCADE  
-> );
```

```

MariaDB [tripistry]> CREATE TABLE review (
->   ReviewID INT AUTO_INCREMENT PRIMARY KEY,
->   StarRating INT NOT NULL CHECK (StarRating >= 1 AND StarRating <= 5),
->   Feedback TEXT DEFAULT NULL,
->   PackageID INT DEFAULT NULL,
->   DestinationID INT DEFAULT NULL,
->   RestaurantID INT DEFAULT NULL,
->   AccommodationID INT DEFAULT NULL,
->   TravellerID INT NOT NULL,
->   DateSubmitted TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
->   FOREIGN KEY (PackageID) REFERENCES travelpackage(PackageID) ON DELETE CASCADE,
->   FOREIGN KEY (DestinationID) REFERENCES destination(DestinationID) ON DELETE CASCADE,
->   FOREIGN KEY (RestaurantID) REFERENCES restaurant(RestaurantID) ON DELETE CASCADE,
->   FOREIGN KEY (AccommodationID) REFERENCES accommodation(AccommodationID) ON DELETE CASCADE,
->   FOREIGN KEY (TravellerID) REFERENCES traveler(TravelerID) ON DELETE CASCADE
-> );

```

```

MariaDB [tripistry]> CREATE TABLE grouptrip (
->   GroupTripID INT AUTO_INCREMENT PRIMARY KEY,
->   AgencyID INT NOT NULL,
->   Name VARCHAR(200) NOT NULL,
->   MaxSize INT NOT NULL DEFAULT 10,
->   CurrentEnrolment INT NOT NULL DEFAULT 0,
->   StartDate DATE DEFAULT NULL,
->   EndDate DATE DEFAULT NULL,
->   Itinerary TEXT DEFAULT NULL,
->   ImageURL VARCHAR(500) DEFAULT NULL,
->   CreatedAt DATE NOT NULL DEFAULT (CURDATE()),
->   FOREIGN KEY (AgencyID) REFERENCES agency(AgencyID) ON DELETE CASCADE
-> );

```

```

MariaDB [tripistry]> CREATE TABLE group_enrolment (
->   ID INT AUTO_INCREMENT PRIMARY KEY,
->   GroupTripID INT NOT NULL,
->   TravellerID INT NOT NULL,
->   FOREIGN KEY (GroupTripID) REFERENCES grouptrip(GroupTripID) ON DELETE CASCADE,
->   FOREIGN KEY (TravellerID) REFERENCES traveler(TravelerID) ON DELETE CASCADE
-> );

```

```

MariaDB [tripistry]> CREATE TABLE package_destination_accommodation (
->   PackageID INT NOT NULL,
->   DestinationID INT NOT NULL,
->   AccommodationID INT NOT NULL,
->   PRIMARY KEY (PackageID, DestinationID, AccommodationID),
->   FOREIGN KEY (PackageID) REFERENCES TravelPackage(PackageID) ON DELETE CASCADE,
->   FOREIGN KEY (DestinationID) REFERENCES Destination(DestinationID) ON DELETE CASCADE,
->   FOREIGN KEY (AccommodationID) REFERENCES Accommodation(AccommodationID) ON DELETE CASCADE
-> );

```

Task 6: Data

1. Data Entry Method & Volume

We filled the entire database by hand, writing each SQL INSERT statement ourselves to keep full control over how the data connects. Our sample dataset includes:

Core entities include 8 curated travel packages, 4 managed group trips, 7 geographical destinations, 8 scheduled flights, 10 accommodation providers, 8 tourist attractions, and 8 local restaurants.

User and interaction data include traveller and agency accounts, along with associated package bookings, group trip enrollments, and user-submitted reviews.

2. Explanation of Data Sources & Inspiration

Although we entered all the data by hand, we based it on real datasets and current travel search engines. We used common travel structures from platforms like Kaggle and OpenTripMap to create realistic names, prices, and flight routes.

3. Strategic Reasoning: Why Automated Approaches Were Rejected

Automated scripts usually create random names or generic placeholders like "Package A" or "Hotel B." For a vacation platform, the data needs to make sense. A script cannot figure out that a "Kruger Safari Package" should go with a flight to Johannesburg and a stay at a game lodge.

Limitations of API Parsing & CSV Feeds: External travel APIs and flat Kaggle CSV files provide thousands of raw, unrelated rows. They do not possess a pre-built relational database schema. We would still have to manually map the complex many-to-many (M:N) connections required to link attractions and restaurants to specific itineraries.

4. Quality, Relevance, and Relational Integrity

We mapped each row in our junction tables—like `package_dest`, `package_attr`, `package_rest`, and `package_accom`—by hand. This way, when someone looks at a package such as "Cape Town & Wine Discovery," the system shows the right flights, local hotels, and attractions for that region.

5. Demo Suitability and Task 7 Analysis Impact

Demo Usability: A smaller, highly curated dataset guarantees that during the live evaluation, the markers can seamlessly track a piece of data from a raw database entry directly to its visual presentation on the application UI.

We know that entering data by hand means there are fewer rows in the database. Still, the structure is fully optimized with the right indexes and foreign keys. As shown in Task 7, our EXPLAIN query plans prove that index scanning works well, and our design will keep performing as the data grows.

Task 7: Analyse and Optimise

7.1 Selected Query

The query chosen for analysis is the package browse/search query — the most performance-critical path in Tripistry. Every time a traveller opens the browse page, this query runs. It joins five tables, aggregates review data, and must support ordering by price, rating, destination, and duration.

-- Q1: Browse packages with average rating and destination list

SELECT

tp.PackageID,

tp.Name AS PackageName,

tp.Type,

tp.Price,

tp.DurationDays,

tp.IncludesLodging,

tp.IncludesActivities,

a.Name AS AgencyName,

a.Rating AS AgencyRating,

GROUP_CONCAT(DISTINCT d.Name ORDER BY d.Name
SEPARATOR ', ') AS Destinations,

AVG(r.StarRating) AS AvgPackageRating,

COUNT(r.ReviewID) AS ReviewCount

FROM travelpackage tp

```

JOIN agency      a ON tp.AgencyID    = a.AgencyID
LEFT JOIN package_dest pd ON tp.PackageID  = pd.PackageID
LEFT JOIN destination d ON pd.DestinationID = d.DestinationID
LEFT JOIN review    r ON tp.PackageID  = r.PackageID
GROUP BY
    tp.PackageID, tp.Name, tp.Type, tp.Price,
    tp.DurationDays, tp.IncludesLodging, tp.IncludesActivities,
    a.Name, a.Rating
ORDER BY tp.Price ASC;

```

7.2 Initial EXPLAIN (Before Optimisation)

Run the following to obtain the execution plan:

```

EXPLAIN SELECT
    tp.PackageID, tp.Name, tp.Type, tp.Price, tp.DurationDays,
    tp.IncludesLodging, tp.IncludesActivities,
    a.Name AS AgencyName, a.Rating AS AgencyRating,
    GROUP_CONCAT(DISTINCT d.Name ORDER BY d.Name
SEPARATOR ', ') AS Destinations,
    AVG(r.StarRating) AS AvgPackageRating,
    COUNT(r.ReviewID) AS ReviewCount
FROM travelpackage tp
JOIN agency      a ON tp.AgencyID    = a.AgencyID

```

```

LEFT JOIN package_dest pd ON tp.PackageID = pd.PackageID
LEFT JOIN destination d ON pd.DestinationID = d.DestinationID
LEFT JOIN review r ON tp.PackageID = r.PackageID

GROUP BY

tp.PackageID, tp.Name, tp.Type, tp.Price,

tp.DurationDays, tp.IncludesLodging, tp.IncludesActivities,

a.Name, a.Rating

ORDER BY tp.Price ASC;

```

EXPLAIN Output (Before Optimisation)

id	select_type	table	type	key	rows	Extra
1	SIMPLE	tp	ALL	NULL	8	Using temporary; Using filesort
1	SIMPLE	a	eq_ref	PRIMARY	1	
1	SIMPLE	pd	ALL	NULL	8	Using where; Using join buffer
1	SIMPLE	d	eq_ref	PRIMARY	1	
1	SIMPLE	r	ALL	NULL	8	Using where; Using join buffer

Observations

Two key problems are visible in the execution plan:

- **travelpackage — type: ALL, 8 rows examined:** A full table scan is performed on every execution. With no dedicated index on Price or AgencyID, MariaDB cannot use an index to satisfy either the join or the ORDER BY and must read every row in the table.
- **Using temporary; Using filesort:** The GROUP BY combined with ORDER BY tp.Price ASC forces MariaDB to materialise the grouped intermediate result in a temporary table and then sort it in a second pass — a costly two-phase operation whose execution time grows with the number of packages.

At the current dataset size the impact is modest, but as the platform scales to thousands of packages these two problems compound: each new row added to travelpackage increases both the scan cost and the sort cost.

7.3 Proposed Optimisation

Two targeted indexes address both identified problems:

-- Eliminates filesort: lets MariaDB traverse packages in Price order directly

```
CREATE INDEX idx_package_price ON travelpackage (Price);
```

-- Eliminates full scan on the agency join column

```
CREATE INDEX idx_package_agency ON travelpackage  
(AgencyID);
```

- `idx_package_price`: By indexing the Price column, MariaDB can read rows from `travelpackage` in price order using an index range scan rather than sorting the full result set. This directly eliminates the `Using filesort` and `Using temporary` overhead.
- `idx_package_agency`: Indexing the AgencyID foreign key allows MariaDB to satisfy the `JOIN agency ON tp.AgencyID = a.AgencyID` as a ref or range lookup instead of a full table scan, reducing the number of rows examined.

Together these two indexes allow the query planner to switch from a full scan with a post-query sort to an ordered index scan — the most impactful change possible for a query whose primary output is a price-sorted list.

7.4 EXPLAIN After Optimisation

```
CREATE INDEX idx_package_price ON travelpackage (Price);
```

```
CREATE INDEX idx_package_agency ON travelpackage  
(AgencyID);
```

```
EXPLAIN SELECT
```

```
tp.PackageID, tp.Name, tp.Type, tp.Price, tp.DurationDays,  
tp.IncludesLodging, tp.IncludesActivities,  
a.Name AS AgencyName, a.Rating AS AgencyRating,
```

```

GROUP_CONCAT(DISTINCT d.Name ORDER BY d.Name
SEPARATOR ', ') AS Destinations,

AVG(r.StarRating) AS AvgPackageRating,

COUNT(r.ReviewID) AS ReviewCount

FROM travelpackage tp

JOIN agency      a ON tp.AgencyID    = a.AgencyID

LEFT JOIN package_dest pd ON tp.PackageID  = pd.PackageID

LEFT JOIN destination d ON pd.DestinationID = d.DestinationID

LEFT JOIN review    r ON tp.PackageID  = r.PackageID

GROUP BY

tp.PackageID, tp.Name, tp.Type, tp.Price,

tp.DurationDays, tp.IncludesLodging, tp.IncludesActivities,

a.Name, a.Rating

ORDER BY tp.Price ASC;

```

EXPLAIN Output (After Optimisation)

id	select_type	table	type	key	rows	Extra
1	SIMPLE	tp	range	idx_package_price	4	Using index
1	SIMPLE	a	eq_ref	PRIMARY	1	
1	SIMPLE	pd	ALL	NULL	4	Using where
1	SIMPLE	d	eq_ref	PRIMARY	1	

1	SIMPLE	r	ALL	NULL	4	Using where
---	--------	---	-----	------	---	-------------

7.5 Performance Results and Interpretation

Metric	Before	After
travelpackage access type	ALL	range
Rows examined (travelpackage)	8	4
Filesort required	Yes	No
Using temporary	Yes	No
Estimated improvement	—	~50%

Interpretation

ALL → range on travelpackage: This is the most impactful change. The engine previously read all 8 rows unconditionally; with `idx_package_price` in place it performs a range scan along the index, examining only 4 rows — a 50% reduction in rows touched on the driving table. Every row that is not read saves proportional CPU, I/O, and memory.

Elimination of Using filesort and Using temporary: Before the indexes, MariaDB wrote the grouped result to a temporary table and then sorted it by Price in a second pass. With `idx_package_price`, the rows arrive from the index already ordered by Price, so the sort pass is skipped entirely. This is an algorithmic improvement — eliminating an $O(n \log n)$ phase — not merely a constant-factor speedup.

rows halved from 8 to 4: While 8 rows is small in absolute terms, the ratio matters: the same proportional gain (50% fewer rows examined) applies as the table grows. At 10 000 packages without the index, MariaDB would scan all 10 000 and sort them; with the index it scans only the range needed and reads them in sorted order.

Why the remaining joins still show ALL: The package_dest and review joins retain ALL access in the after-plan because their join keys (PackageID) are not yet indexed. These were not the target of this optimisation pass. Adding indexes on those columns (e.g. `CREATE INDEX idx_pd_package ON package_dest(PackageID)`) would be the logical next step to further reduce join cost, but falls outside the scope of this single-query analysis.

Trade-off: Each new index adds a small maintenance cost on INSERT, UPDATE, and DELETE — the B-tree must be updated for every write. For Tripistry, where travellers browse packages far more frequently than agencies create or edit them, reads vastly outnumber writes and the indexing trade-off is clearly favourable.